FastAPI Reference

`Response` class

You can declare a parameter in a *path operation function* or dependency to be of type `Response` and then you can set data for the response like headers or cookies.

You can also use it directly to create an instance of it and return it from your *path operations*.

Read more about it in the FastAPI docs about returning a custom Response

You can import it directly from `fastapi`:

```
from fastapi import Response
```

class fastapi.Response

```
Response(
    content=None,
    status_code=200,
    headers=None,
    media_type=None,
    background=None,
)
```

Source code in `starlette/responses.py`

```
def __init__(
    self,
    content: Any = None,
    status_code: int = 200,
    headers: Mapping[str, str] | None = None,
    media_type: str | None = None,
    background: BackgroundTask | None = None,
) -> None:
    self.status_code = status_code
    if media_type is not None:
        self.media_type = media_type
    self.background = background
    self.body = self.render(content)
    self.init_headers(headers)
```

attr media_type `class-attribute` `instance-attribute`

```
media_type = None
```

attr charset `class-attribute` `instance-attribute`

```
charset = 'utf-8'
```

attr status_code `instance-attribute`

Ask AI

```
status_code = status_code
```

attr background `instance-attribute`

```
background = background
```

attr body `instance-attribute`

```
body = render(content)
```

attr headers `property`

```
headers
```

meth render

```
render(content)
```

Source code in `starlette/responses.py`

```
def render(self, content: Any) -> bytes | memoryview:
    if content is None:
        return b""
    if isinstance(content, bytes | memoryview):
        return content
    return content.encode(self.charset)  # type: ignore
```

meth init_headers

```
init_headers(headers=None)
```

Source code in `starlette/responses.py`

```
def init_headers(self, headers: Mapping[str, str] | None = None) -> None:
    if headers is None:
        raw_headers: list[tuple[bytes, bytes]] = []
        populate_content_length = True
        populate_content_type = True
    else:
        raw_headers = [(k.lower().encode("latin-1"), v.encode("latin-1")) for k, v in
headers.items()]
        keys = [h[0] for h in raw_headers]
        populate_content_length = b"content-length" not in keys
        populate_content_type = b"content-type" not in keys

    body = getattr(self, "body", None)
    if (
        body is not None
        and populate_content_length
        and not (self.status_code < 200 or self.status_code in (204, 304))
    ):
        content_length = str(len(body))
        raw_headers.append((b"content-length", content_length.encode("latin-1")))

    content_type = self.media_type
    if content_type is not None and populate_content_type:
        if content_type.startswith("text/") and "charset=" not in content_type.lower():
            content_type += "; charset=" + self.charset
        raw_headers.append((b"content-type", content_type.encode("latin-1")))

    self.raw_headers = raw_headers
```

`meth` set_cookie

```
set_cookie(
    key,
    value="",
    max_age=None,
    expires=None,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
    partitioned=False,
)
```

Source code in `starlette/responses.py`

```python
def set_cookie(
    self,
    key: str,
    value: str = "",
    max_age: int | None = None,
    expires: datetime | str | int | None = None,
    path: str | None = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
    partitioned: bool = False,
) -> None:
    cookie: http.cookies.BaseCookie[str] = http.cookies.SimpleCookie()
    cookie[key] = value
    if max_age is not None:
        cookie[key]["max-age"] = max_age
    if expires is not None:
        if isinstance(expires, datetime):
            cookie[key]["expires"] = format_datetime(expires, usegmt=True)
        else:
            cookie[key]["expires"] = expires
    if path is not None:
        cookie[key]["path"] = path
    if domain is not None:
        cookie[key]["domain"] = domain
    if secure:
        cookie[key]["secure"] = True
    if httponly:
        cookie[key]["httponly"] = True
    if samesite is not None:
        assert samesite.lower() in [
            "strict",
            "lax",
            "none",
        ], "samesite must be either 'strict', 'lax' or 'none'"
        cookie[key]["samesite"] = samesite
    if partitioned:
        if sys.version_info < (3, 14):
            raise ValueError("Partitioned cookies are only supported in Python 3.14 and above.")
# pragma: no cover
        cookie[key]["partitioned"] = True  # pragma: no cover

    cookie_val = cookie.output(header="").strip()
    self.raw_headers.append((b"set-cookie", cookie_val.encode("latin-1")))
```

`meth` delete_cookie

```python
delete_cookie(
    key,
    path="/",
    domain=None,
    secure=False,
    httponly=False,
    samesite="lax",
)
```

Source code in `starlette/responses.py`

```python
def delete_cookie(
    self,
    key: str,
    path: str = "/",
    domain: str | None = None,
    secure: bool = False,
    httponly: bool = False,
    samesite: Literal["lax", "strict", "none"] | None = "lax",
) -> None:
    self.set_cookie(
        key,
        max_age=0,
        expires=0,
        path=path,
        domain=domain,
        secure=secure,
        httponly=httponly,
        samesite=samesite,
    )
```